



GLOBALPAY

Mandatory Architecture & Engineering Deliverables

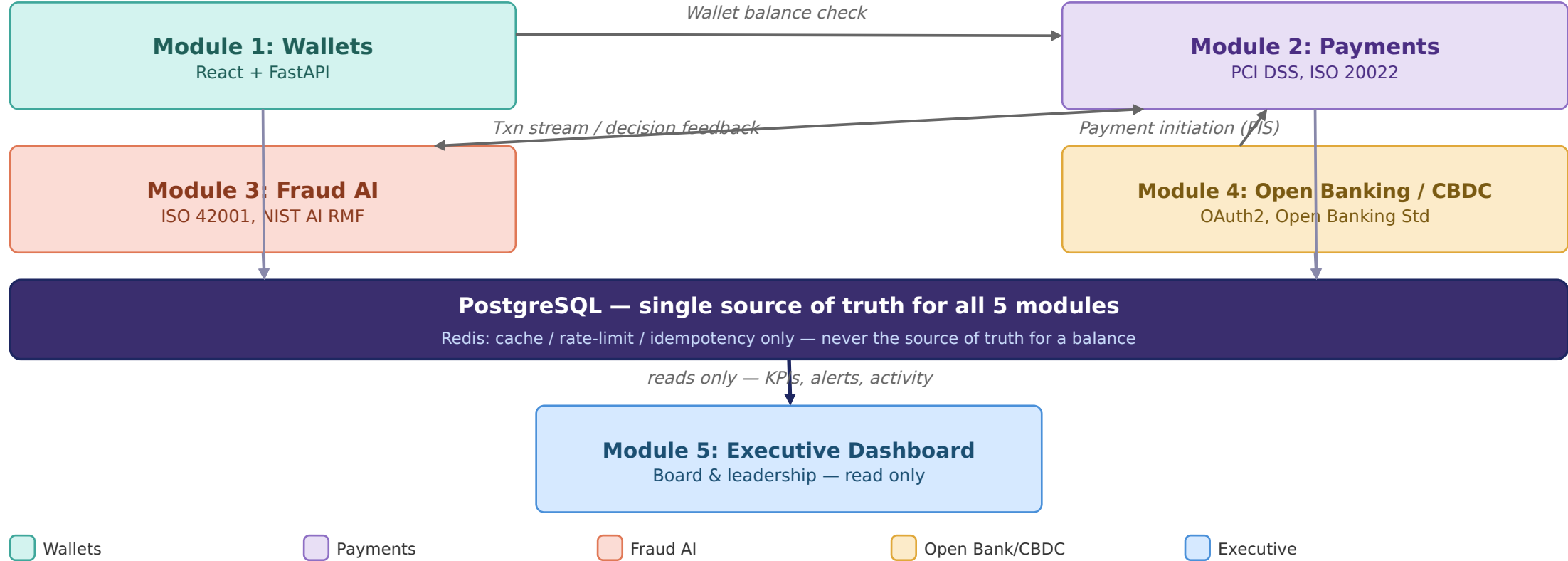
All 17 mandatory diagrams, numbered exactly as listed in the exam brief

#	Diagram	Also in Stage 1 deck?
1	Enterprise Financial Technology Architecture	Yes
2	High-Level Solution Architecture	No - here only
3	Detailed System Architecture	No - here only (composite index)
4	Digital Wallet Architecture	Yes (Module 1)
5	Payment Processing Architecture	Yes (Module 2)
6	Open Banking API Architecture	Yes (Module 4)
7	AI Fraud Detection Architecture	Yes (Module 3)
8	CBDC Simulation Architecture	No - here only
9	Enterprise Network Architecture	Yes
10	Network Flow Diagram	Yes (combined in Module 11)
11	Data Flow Diagram (Level 0)	Yes (combined)
12	Data Flow Diagram (Level 1)	Yes (combined)
13	Digital Payment Workflow	Yes (combined in Module 2)
14	Fraud Detection Workflow	Yes (combined in Module 3)
15	Open Banking API Workflow	Yes (combined in Module 4)
16	Database ERD	Yes
17	Sequence Diagram	No - here only

G Enterprise Financial Technology Architecture

1

How the five modules connect

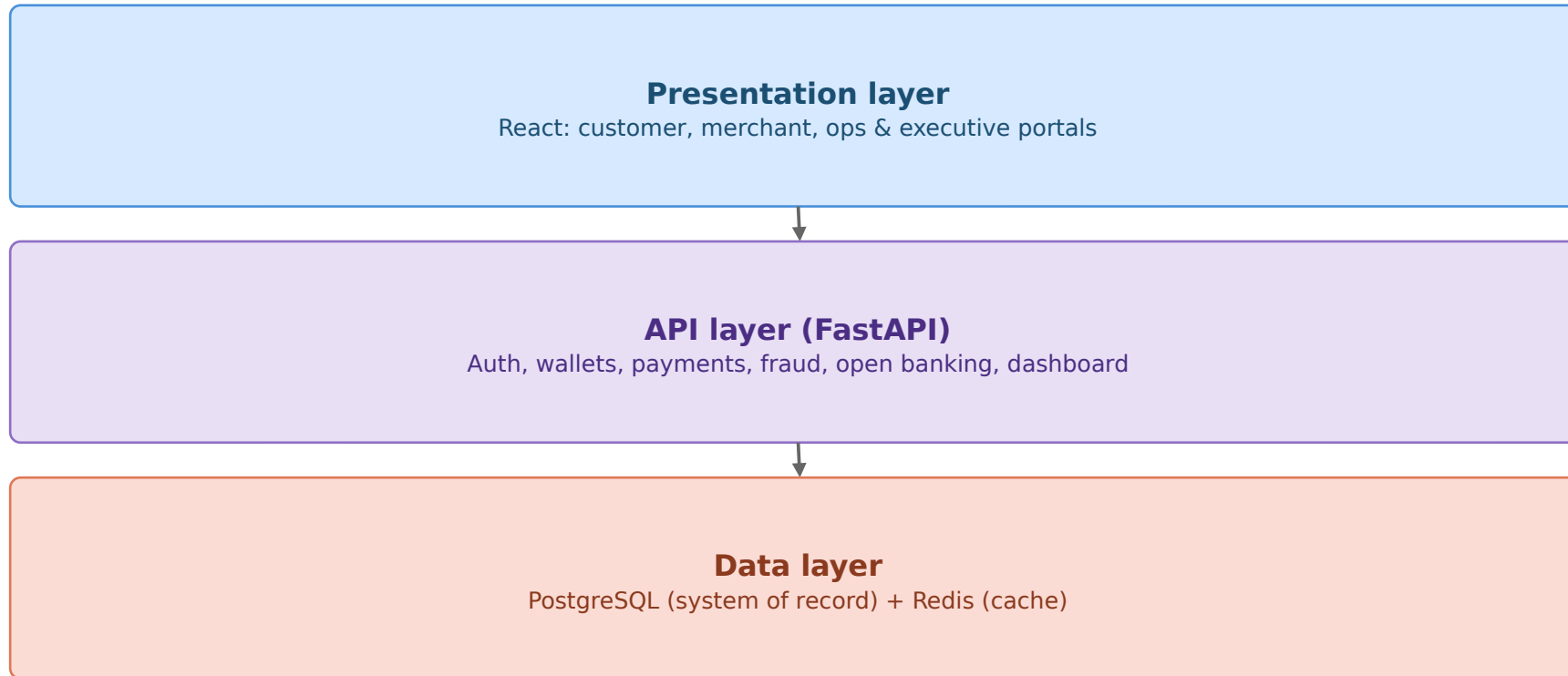


Modules 1-4 write to PostgreSQL; Module 5 reads only; only the named service calls above cross module boundaries

G High-Level Solution Architecture

Layered platform view

2



Security & compliance

ISO 27001 / 27017 / 27018
PCI DSS v4.0*
OWASP Top 10 + API Top 10
ISO 42001 / NIST AI RMF
NIST CSF 2.0 + CIS Controls
COBIT 2019

Security and compliance controls apply across the presentation, API and data layers

G Detailed System Architecture

3

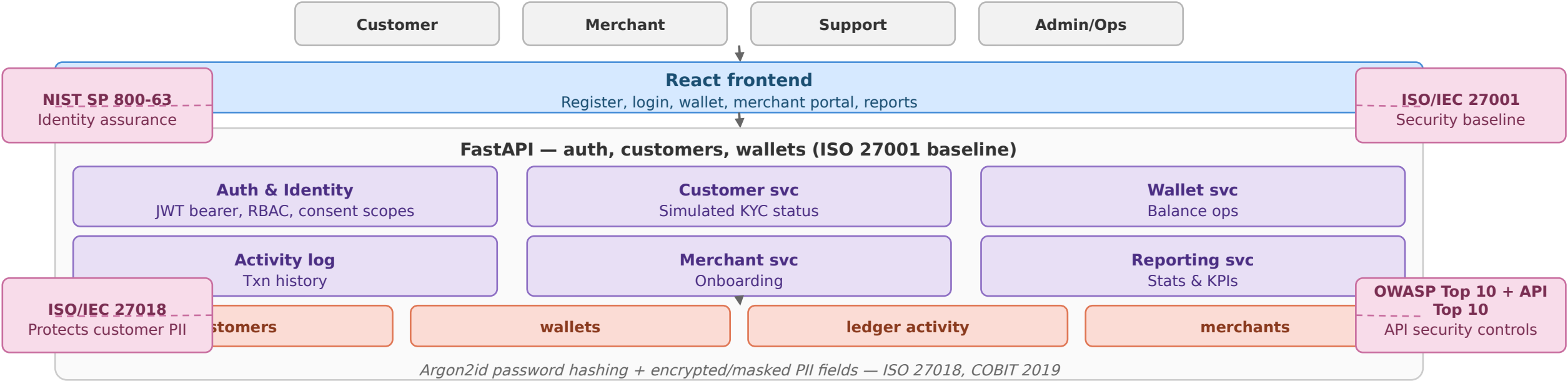
Five implementation modules and their primary responsibilities

Module 1 Wallets	React UI -> FastAPI (auth, customers, wallets, merchants) -> PostgreSQL
Module 2 Payments	Validation, transaction engine, double-entry ledger -> ledger tables
Module 3 Fraud AI	Feature engineering -> Logistic Regression -> risk scoring -> human review
Module 4 Open Banking/CBDC	AIS/PIS APIs, CBDC API, activity monitoring -> separate CBDC ledger
Module 5 Executive Dashboard	Read-only aggregator across Modules 1-4 -> executive reports

Each module follows the shared React, FastAPI and PostgreSQL architecture with common security, audit and reporting controls

G Digital Wallet Architecture

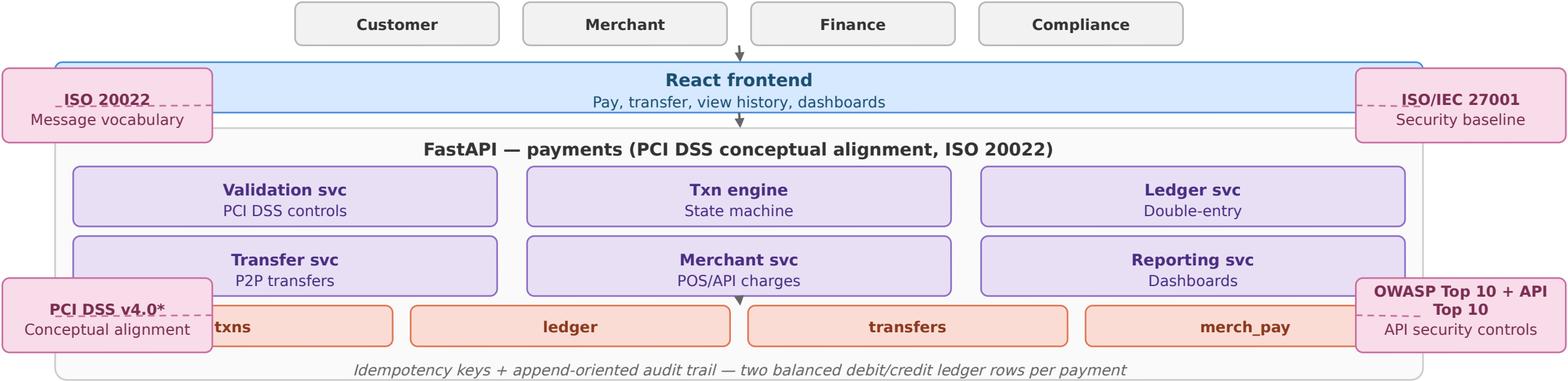
Registration, wallet creation, balance operations, merchant accounts



Wallet and identity operations write directly to PostgreSQL

G Payment Processing Architecture

Validation, double-entry ledger, transfers, merchant charges

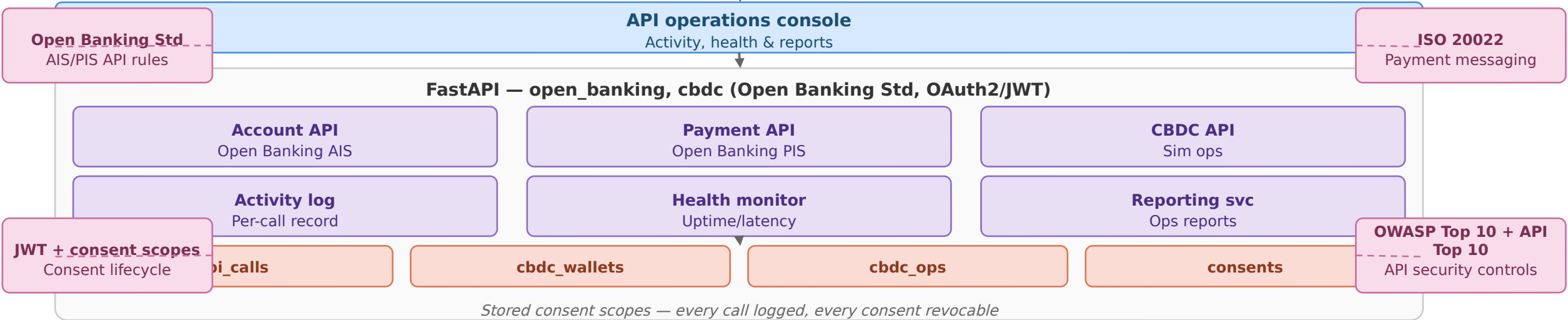


Direct link to Module 3 for fraud scoring — the only true module-to-module call besides Module 4's PIS

G Open Banking API Architecture

6

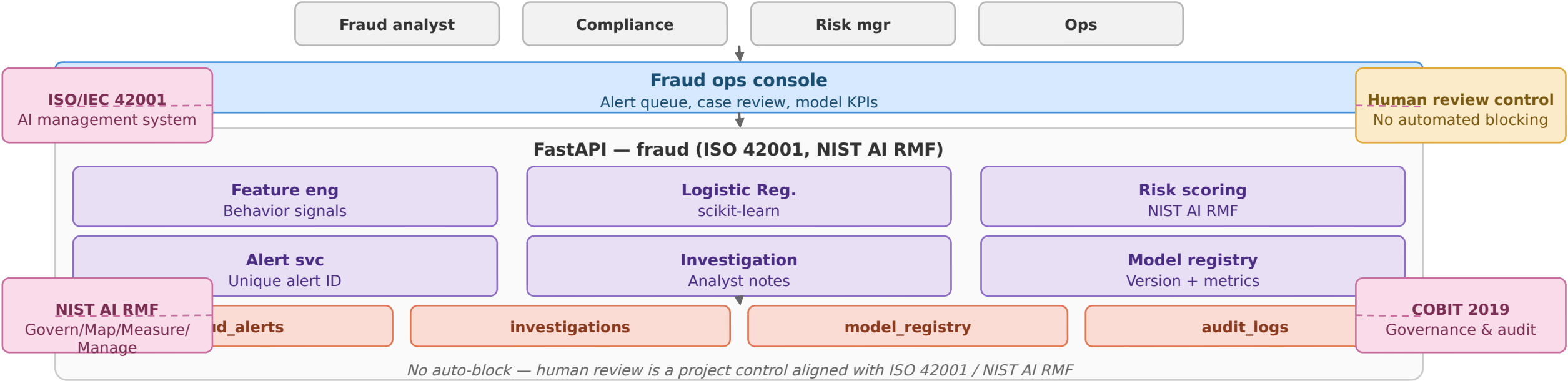
Account info & payment initiation APIs, CBDC wallet ops



PIS reuses Module 2 transfer logic; CBDC wallet and operation records remain separate from commercial wallets

G AI Fraud Detection Architecture

Feature engineering, classification, mandatory human review



Surfaces to fraud/risk/compliance teams only — never shown to customers

G CBDC Simulation Architecture

8

Kept separate from the commercial wallet ledger

OAuth2/JWT, ISO 20022
Access + messaging

COBIT 2019
Governance & audit



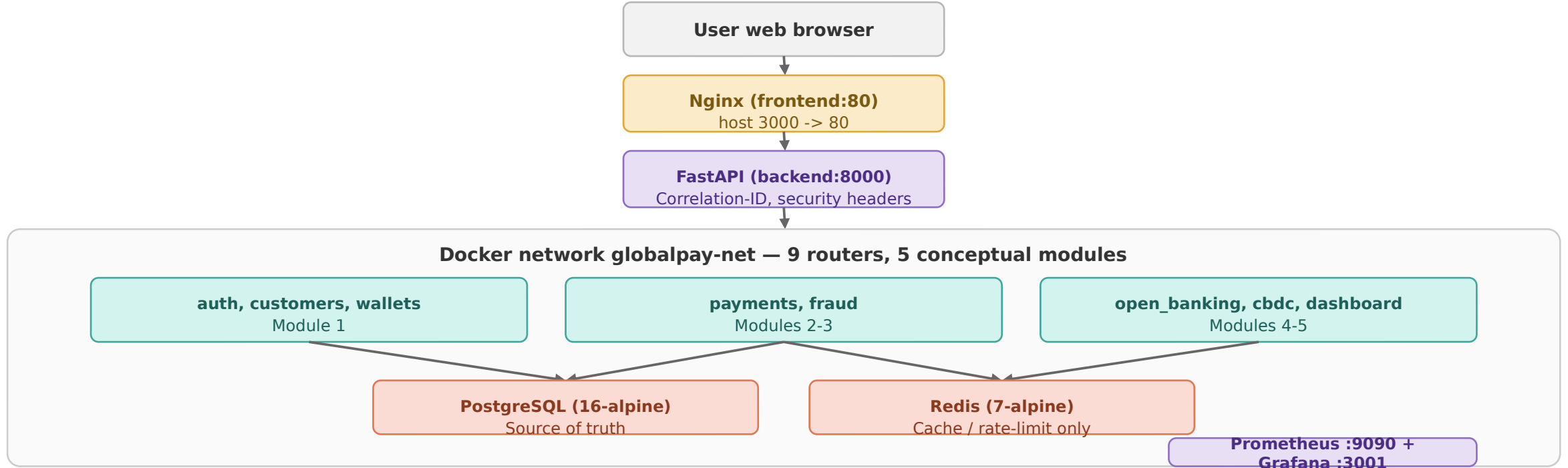
Reporting reads `cbdc_wallets` and `cbdc_operations`; these records remain separate from commercial wallet data

Central-bank money is never mixed with commercial wallet balances — this table separation is the audit boundary

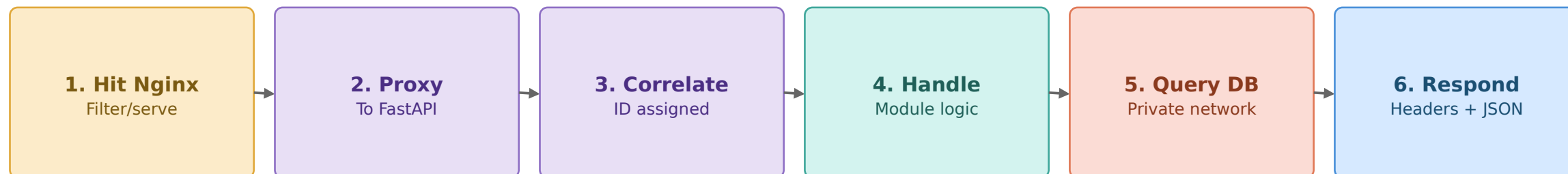
G Enterprise Network Architecture

9

compose.yaml — 6 containers on one Docker network (globalpay-net)



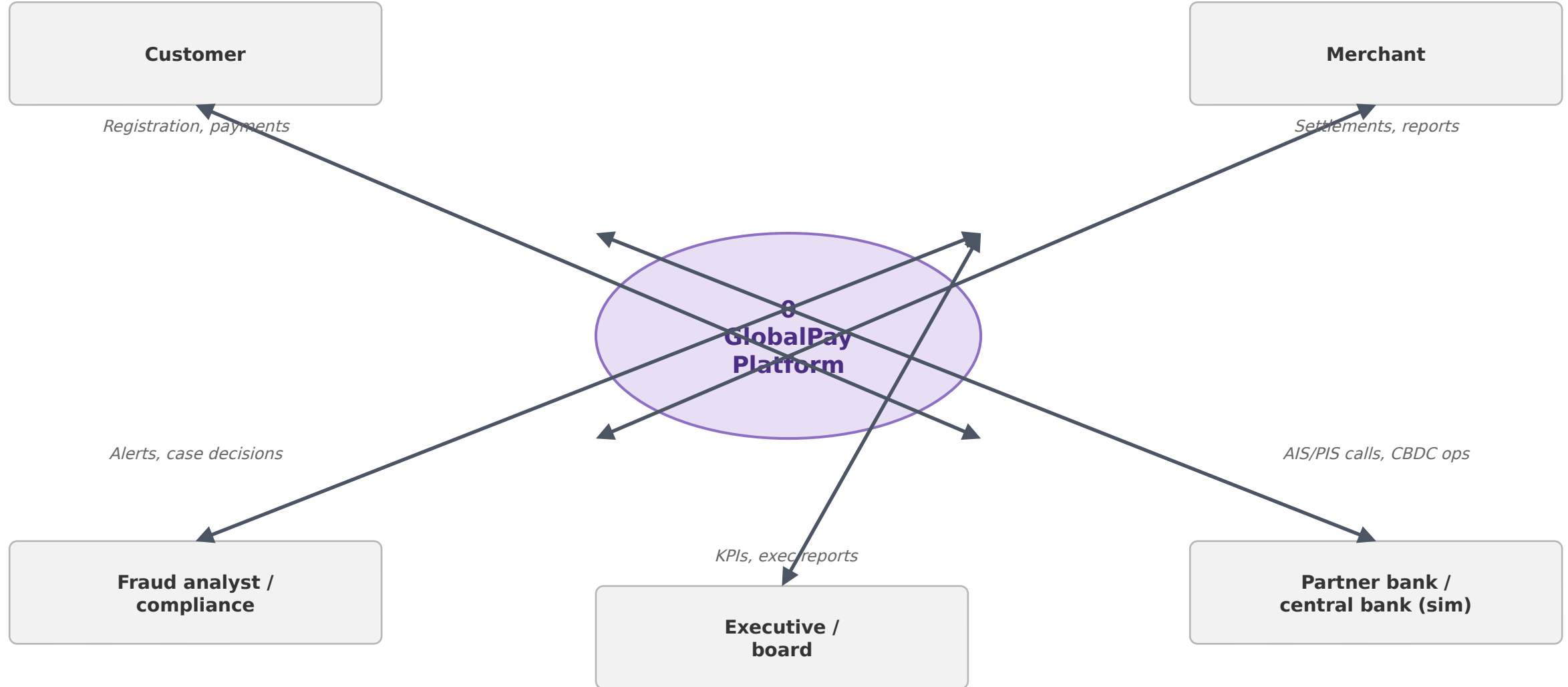
Redis never holds the authoritative wallet balance — Postgres is the single source of truth



G Data Flow Diagram — Level 0 (Context)

11

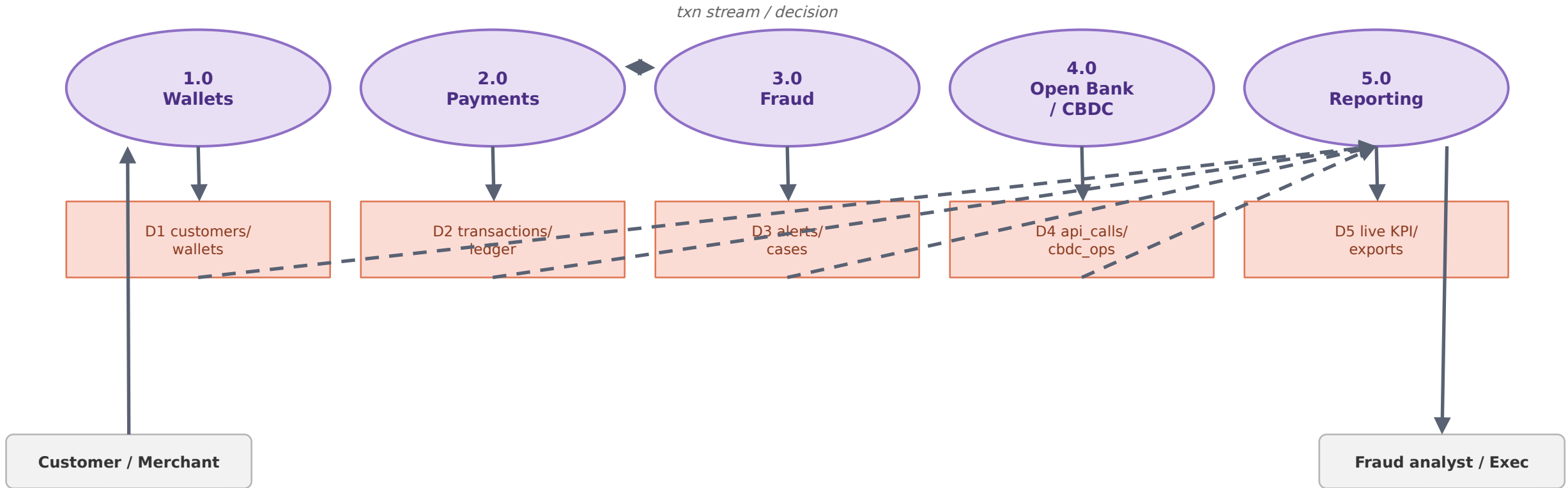
The platform as a single process, with its external entities



G Data Flow Diagram — Level 1

12

The platform decomposed into its five core processes

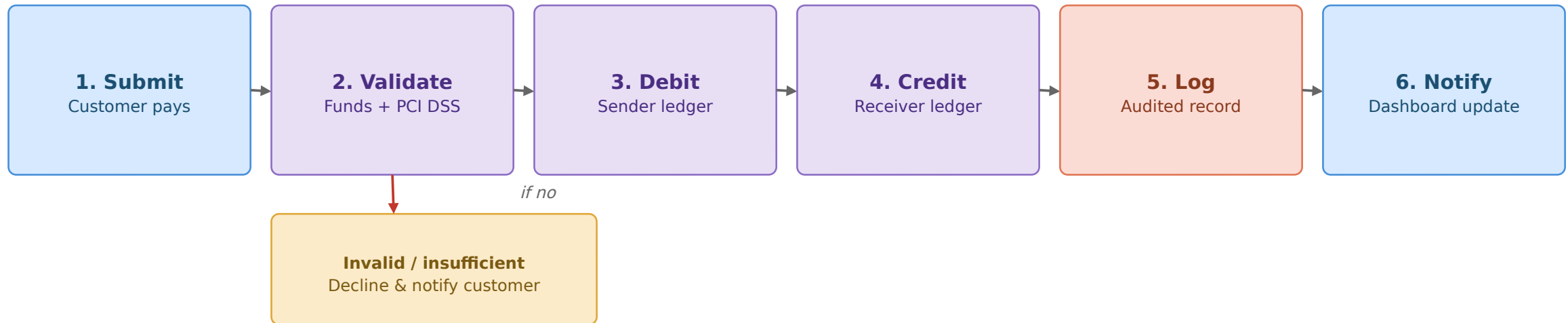


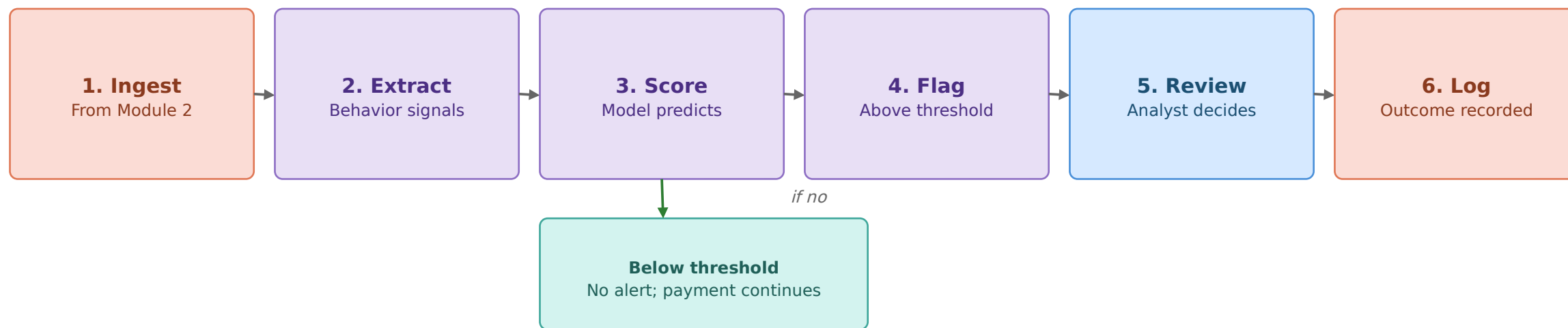
Dashed arrows: on-demand aggregation from data stores into 5.0 Reporting

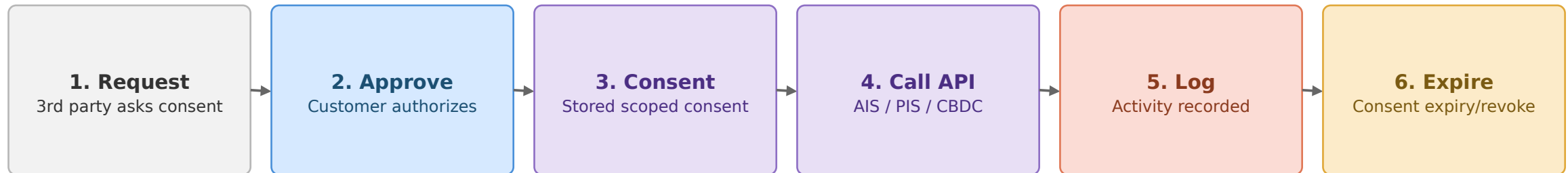
G Digital Payment Workflow

Validation branch shown explicitly

13



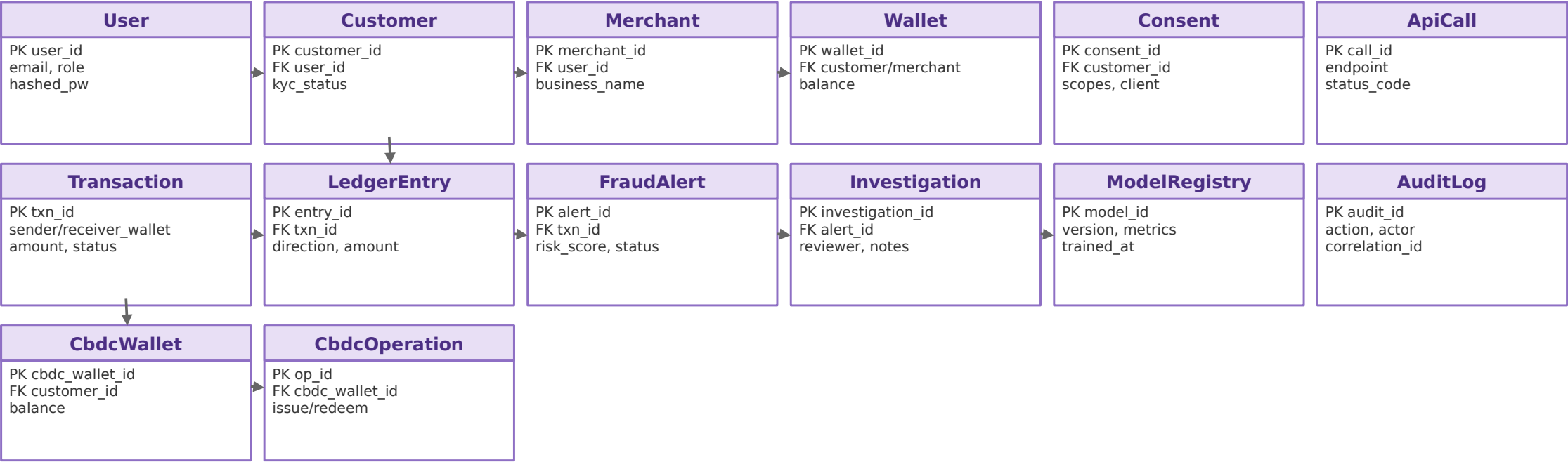




G

Database Entity Relationship Diagram (ERD)

14 tables from backend/app/models/entities.py



User -> {Customer, Merchant}; Customer -> {Wallet, Consent, CbdcWallet}; Merchant -> Wallet. Transaction -> {LedgerEntry, FraudAlert -> Investigation}. ModelRegistry and AuditLog are cross-cutting.

End-to-end lifecycle across all five modules

